

Task 2 – Semantic Equivalence in Formal and Informal Persian

Natural language allows the same core meaning to be expressed in a wide variety of structural forms. This phenomenon is also observed in the Persian language, where informal conversational speech exhibits profound phonological, lexical, and syntactic deviations from its formal written counterpart. While humans effortlessly bridge these stylistic variations to recognize semantic equivalence, enabling computational models to capture abstract meaning beyond surface-level token alignment remains a hurdle in Persian Natural Language Processing (NLP).

In this assignment, you will examine the computational mechanics of semantic representation. Moving from traditional distributed word representations to state-of-the-art context-aware transformers, you will build an end-to-end data-centric pipeline to identify semantic alignment across stylistic boundaries.

Dataset Description

This project utilizes the [ParsMap](#) dataset, a curated corpus containing pairs of semantically equivalent formal and informal Persian sentences. The dataset is structured to facilitate an empirical comparison between surface lexical adjustments and core semantic consistency. It contains two columns, namely `formalForm` and `informalForm`, each holding a string. You will use this dataset across all phases to analyze linguistic patterns, measure embedding similarities, and train a supervised transformer model.

1. Exploratory Data Analysis and Linguistic Properties (29 Points)

Raw textual data must be comprehensively understood before feeding it into any modeling architecture. In this section, you will dissect the structural and vocabulary discrepancies inherent to formal and informal Persian.

1.1 Dataset Exploration & Transformation Mapping (8 Points)

Before executing structural counts, conduct a qualitative review of stylistic shift dynamics within the dataset.

- Load the dataset into a pandas DataFrame and sample a collection of random sentence pairs.
- Display both the formal and informal variants side by side.
- Identify and document specific linguistic modifications occurring during the formal-to-informal shift.

1.2 Sentence Length Distribution Analysis (7 Points)

Quantify structural differences by measuring sequence lengths across both domains.

- For every sentence in the corpus, calculate two distinct metrics: Word Count (via whitespace splitting) and Character Count.
- Calculate the statistical metrics for word count and character count (mean, median, standard deviation, etc.) for formal and informal sentences, and compare the results. Justify any differences in the values.
- (Optional) Generate comparative visualizations contrasting the statistical distributions for both formal and informal subsets. You can use overlaying Histograms, Kernel Density Estimate (KDE) plots, and Boxplots to isolate outliers.

1.3 Stylistic Vocabulary Analysis (7 Points)

Examine the overlap and uniqueness of lexicons between the two textual registers.

- Construct two separate vocabularies representing the formal and informal corpora.
- Compute total vocabulary sizes, isolate words unique to each register, and extract the top 20 most frequent terms for each category.
- Compare the vocabulary size of formal and informal corpora and justify the differences. Which corpora has a larger vocabulary? Why?

1.4 Transformer Tokenization Complexity (7 Points)

Modern subword tokenization schemes process sequences differently than classical whitespace splits, especially when dealing with non-standard dialects or casual scripts.

- Load the native tokenizer associated with the pre-trained ParsBERT architecture (HooshvareLab/bert-fa-base-uncased).
- Tokenize the data once using whitespace splitting and once with the ParsBert tokenizer.

- Compare subword fragmentation against standard whitespace tokenization across a number of selected sentence pairs. Spot the differences.

Token Inflation Factor (TIF): To mathematically isolate the tokenization overhead introduced by casual language variants, calculate the ratio:

$$\text{TIF} = \frac{\text{Number of Transformer Tokens}}{\text{Number of Whitespace Tokens}}$$

- Compute the TIF for every sentence in the dataset. Plot and compare the TIF distributions for both the formal and informal pools. Which one has a higher TIF? Why?

2. Representing Meaning with Semantic Embeddings (38 Points)

Vector spaces allow us to map text into continuous representations where distance correlates with semantic similarity. In this section, you will evaluate whether static and contextual embeddings can recognize that formal and informal expressions carry identical semantics.

2.1 Static Word Representations via Word2Vec (10 Points)

- Research and understand the Word2Vec model, including its underlying principles, training objectives, and the differences between the Continuous Bag-of-Words (CBOW) and Skip-Gram architectures. Understand how Word2Vec learns distributed word representations and captures semantic relationships between words.
- Utilize the `gensim.models.Word2Vec` library to train a continuous skip-gram or CBOW model over the entire integrated ParsMap text corpus.
- Select an explicit set of target vocabulary terms (preferably among the more frequent words) containing both formal keywords and their informal equivalents. Extract their top 5 nearest neighbors using Cosine Similarity.
- Examine whether formal and informal expressions converge into shared semantic spaces.

2.2 Sentence Similarity & Evaluation Framework (11 Points)

- Construct a sentence-level embedding function by computing the element-wise average of constituent Word2Vec token vectors for every sentence.
- Formulate two diagnostic evaluation groups:
 - **Group A (Positive Controls):** True aligned formal-informal pairs extracted from the dataset.
 - **Group B (Negative Controls):** Corrupted pairs created by matching formal sentences with completely random informal sentences.
- Compute the Cosine Similarity score for every pair across both groups. Plot the resulting distributions for Group A and Group B on a single chart.
- Find some equivalent pairs that have a Cosine Similarity score less than 0.4 and print them out. Can you justify why Word2Vec has assigned a low similarity score to them?

2.3 Contextual Representations with ParsBERT (11 Points)

- Initialize a context-aware transformer by loading the pre-trained ParsBERT model (HooshvareLab/bert-fa-base-uncased) via the Hugging Face transformers library.
- Extract dense sentence representations. You may use the pooled output ([CLS] token embedding) or apply mean pooling across the final hidden state sequence layer.
- Re-run the exact similarity experiment outlined in Task 2.2 using these contextual vectors. Compute similarity scores for Group A and Group B, and plot their distributions.
- Compare the similarity distributions of ParsBERT and Word2Vec. Justify any differences.

2.4 High-Dimensional Space Visualization (6 Points)

- Reduce high-dimensional sequence embeddings into a two-dimensional plane using Principal Component Analysis (PCA) alongside non-linear reduction techniques (t-SNE or UMAP).
- Generate separate 2D scatter plots for the Word2Vec and ParsBERT embedding spaces. Use different colors to distinguish formal and informal sentence representations. For 30 randomly selected formal-informal sentence pairs, draw straight lines connecting the corresponding points to visualize the distance between equivalent expressions in each embedding space.

- Compare and interpret the two visualizations. Discuss the difference between the two plots and the overall clustering patterns. How do they correspond to the dynamics of Word2Vec and ParsBERT?
- Based on the plots, determine which model (Word2Vec or ParsBERT) better aligns semantically equivalent formal and informal expressions. Justify your answer.

3. Supervised Semantic Equivalence Classification (43 Points)

While raw embedding distance offers an unsupervised metric for semantic proximity, fine-tuning a transformer model allows its multi-head attention layers to learn the explicit mapping of stylistic shifts. In this section, you will optimize ParsBERT for a sequence-pair classification task.

3.1 Binary Classification Dataset Construction (9 Points)

Convert the parallel corpus into a supervised learning framework.

- **Positive Class (Label 1):** Aligned formal-informal pairs from the original dataset (True Semantic Equivalents).
- **Negative Class (Label 0):** Unaligned pairs generated by matching formal sentences with non-equivalent, random informal entries. Maintain a strict 1:1 class balance.
- **Data Leakage Precaution:** To avoid data leakage, ensure that no individual sentence appears in more than one split (train/val/test) in any role (positive or negative pair). Shuffle the sentences and assign them to splits *before* generating negative pairs.
- Split the generated dataset into separate Training (80%), Validation (10%), and Testing (10%) partitions. Ensure no cross-contamination or data leakage occurs between splits.

3.2 Transformer Input Encoding (8 Points)

Transformers require highly structured inputs to process sentence pairs simultaneously.

- Pass pairs through the ParsBERT tokenizer to generate proper model inputs.
- Extract and display a single tokenized sample. Document and explain the operational role of the following arrays:
 - `input_ids` (including the placement of special tokens like [CLS] and [SEP]).

- attention_mask (handling padding variations across batches).
- token_type_ids / segment_ids (distinguishing sequence A from sequence B).

3.3 Fine-Tuning ParsBERT (16 Points)

Fine-tune the architecture to resolve semantic equivalence under style variations.

- Append a binary classification head on top of the pre-trained ParsBERT model (AutoModelForSequenceClassification).
- Configure optimization hyperparameters: set an appropriate learning rate (e.g., 2×10^{-5} to 5×10^{-5}), employ an AdamW optimizer, utilize a linear scheduler with warmup, and select a manageable batch size.
- Train the architecture across multiple epochs using available hardware (Kaggle, Google Colab, or local GPU resources). Track training loss and validation loss at regular step intervals.
- Plot training and validation loss curves to monitor convergence and guard against overfitting.

3.4 Evaluation, Success Analysis, and Live Inference (10 Points)

Evaluate the model on the test partition, examine its successes and failures, and implement an interactive evaluation tool.

- **Quantitative Evaluation:** Execute inference over the held-out test split. Compute comprehensive classification performance metrics: Accuracy, Precision, Recall, and F1-Score (macro-averaged).
- **Success Analysis:** Isolate and investigate several correctly classified samples from the test set. Examine both *True Positives* (where equivalent pairs were accurately matched) and *True Negatives* (where random pairings were accurately rejected). Analyze the semantic stability or vocabulary cues that facilitated these successful predictions despite register shifts.
- **Error Analysis:** Isolate several incorrectly classified examples (*False Positives* and *False Negatives*). Display the text pairs alongside their true labels and predicted probabilities, discussing possible reasons for model confusion.
- **Live Inference Function:** Develop a standalone, interactive Python function:

```
def predict_equivalence(formal_sentence: str, informal_sentence: str) -> dict:
    """
    Receives a formal and an informal Persian sentence string,
    tokenizes them as a sentence pair, performs inference using
    the fine-tuned ParsBERT model, and returns the equivalence
    decision along with the confidence score.
    """
    pass
```

Demonstrate the utility of this function by testing it on novel, out-of-dataset sentence pairs that you craft yourself.

Final Discussion

As a final discussion and conclusion, reflect on the following questions and ensure you can answer them, but there's no need to include the answers in your report.

1. What are the primary structural, morphological, and syntactic differences between formal and informal Persian discovered during your exploration?
2. Why does the averaging of static word embeddings (Word2Vec) encounter structural limitations when mapping sentences across different stylistic domains?
3. What specific architectural components enable pre-trained transformers (ParsBERT) to successfully capture semantic alignment despite extensive lexical changes?
4. How does the process of supervised fine-tuning modify the model's inner mechanics compared to utilizing its frozen, pre-trained representations? Investigate the differences between the tokenization approach utilized in sections 2 and 3, if any.